Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 7.5

Obtained Marks:

Artificial Neural Networks

Lab Project

Last Date of Submission: 20th October, 2026

MomentMind AI: Natural Language Video Moment Retrieval

Submitted To: Muhammad Hasaan Mujtaba

Student Name: Jalisha Khalid Ali

Registration Number: 24108209

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

1. Project Overview

MomentMind AI is a natural-language video retrieval system that helps users find visually relevant moments without manually watching an entire video. A user uploads a video, enters a text query describing an action or scene, and receives up to five relevant sampled frames with timestamps and similarity scores.

2. Objectives

- Search video content using ordinary text, such as "a person opens a door."
- Rank sampled frames according to their similarity to the text query.
- Display the top five matching frames, timestamps, similarity scores, and blur scores.
- Provide an estimated time window around the best-matching frame.
- Run the application as a local website on a Windows computer.

3. Technologies Used

Technology	Purpose
Python	Main programming language and application logic
Google Colab	Development and GPU-based experimentation
PyTorch + Transformers	Loading and running the pretrained CLIP model
CLIP (openai/clip-vit-base-patch32)	Comparing text descriptions with video frames
OpenCV	Reading videos, sampling frames, and calculating blur scores
Flask	Backend web application and upload handling
HTML / CSS	Website structure and visual design

4. System Workflow

- The user uploads a supported video through the website.
- OpenCV reads the video and samples a frame approximately every 0.5 seconds.
- The sampled frames are prepared as images for CLIP.
- The text query is evaluated in three forms: the original query and two descriptive prompt variations.
- CLIP calculates text-image similarity scores for the sampled frames.
- The frames are ranked, and up to five top results are selected.
- The website displays matching frames, timestamps, scores, and a short report.

5. Main Features

- Natural-language search for actions and scenes.
- Top-five matching-frame gallery with timestamps and similarity scores.
- Approximate time window around the best-ranked frame.
- Blur analysis using variance of the Laplacian, with a rough low-clarity indicator.
- Flask website with custom HTML/CSS styling.
- Windows launcher (run_windows.bat) to help install dependencies and start the app.

6. Implementation

Development began in Google Colab, where the pretrained CLIP model was loaded and tested on an uploaded video. OpenCV extracted frames at 0.5-second intervals and calculated a blur score for each frame. A text query was compared with the sampled images, and the highest-scoring frames were displayed. A Gradio prototype was used to test video upload and search.
The prototype was then implemented as a Flask web application. The backend handles uploads, frame extraction, CLIP similarity ranking, and result generation. The frontend uses HTML and CSS for the upload form, search box, results gallery, and report. The project was packaged into a ZIP with the required files and a Windows launcher so it can run locally from CMD.

7. Results and Testing

The application was tested successfully with uploaded videos. It returned up to five ranked frames for a natural-language query and displayed timestamps, similarity scores, blur information, and an estimated time window. The Flask website was launched on a Windows laptop and the results were displayed in the browser.

8. Future Enhancements

- Improve temporal localization to estimate complete action boundaries.
- Add more robust codec handling and playable result clips.
- Evaluate retrieval accuracy on a larger set of videos and queries.
- Improve blur warnings and investigate optional frame enhancement.
- Deploy the website for access from other computers, if required.

9. Code:

Loading the pretrained CLIP model

```
import torch
import cv2
import gradio as gr
import transformers

print("All libraries imported successfully!")
print("PyTorch:", torch.__version__)
print("Transformers:", transformers.__version__)
print("OpenCV:", cv2.__version__)
print("Gradio:", gr.__version__)
print("GPU ready:", torch.cuda.is_available())
```

```
All libraries imported successfully!
PyTorch: 2.11.0+cu130
Transformers: 5.18.0
OpenCV: 4.14.0
Gradio: 6.29.0
GPU ready: True
```

```python
import torch
from transformers import CLIPModel, CLIPProcessor

# Select the pretrained CLIP model
model_name = "openai/clip-vit-base-patch32"

# Use GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"

print("Loading pretrained CLIP model...")
print("Selected device:", device)

# Load the model and its processor
processor = CLIPProcessor.from_pretrained(model_name)
model = CLIPModel.from_pretrained(model_name)

# Move the model to the selected device
model = model.to(device)

# Set inference mode (no training)
model.eval()

print("CLIP loaded successfully!")
print("Device:", next(model.parameters()).device)
```

```
Loading pretrained CLIP model...
Selected device: cuda
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
preprocessor_config.json: 100% 316/316 [00:00<00:00, 9.40kB/s]
config.json: 100% 4.19k/4.19k [00:00<00:00, 90.9kB/s]
tokenizer_config.json: 100% 592/592 [00:00<00:00, 17.2kB/s]
vocab.json: 100% 862k/862k [00:00<00:00, 11.3MB/s]
merges.txt: 100% 525k/525k [00:00<00:00, 6.27MB/s]
tokenizer.json: 100% 2.22M/2.22M [00:00<00:00, 17.9MB/s]
special_tokens_map.json: 100% 389/389 [00:00<00:00, 8.68kB/s]
pytorch_model.bin: reconstructing file: 100% 605MB / 605MB, 50.4MB/s
pytorch_model.bin: downloading bytes: 433MB, 35.3MB/s
Loading weights: 100% 398/398 [00:00<00:00, 5587.76it/s]
CLIP loaded successfully!
Device: cuda:0
```

Video upload and frame extraction

```
import cv2
import os

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise ValueError("Video open nahi ho rahi. Please upload another video.")

fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

duration = total_frames / fps if fps > 0 else 0

print("File:", os.path.basename(video_path))
print("FPS:", fps)
print("Total frames:", total_frames)
print("Resolution:", width, "x", height)
print("Duration:", round(duration, 2), "seconds")

cap.release()
```

```
File: 8747616-uhd_2160_3840_25fps.mp4
FPS: 25.0
Total frames: 249
Resolution: 2160 x 3840
Duration: 9.96 seconds
```

```
import matplotlib.pyplot as plt

cap = cv2.VideoCapture(video_path)
success, frame = cap.read()
cap.release()

if success:
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(8, 5))
    plt.imshow(frame_rgb)
    plt.title("First Frame of Uploaded Video")
    plt.axis("off")
    plt.show()
else:
    print("Could not read the first frame.")
```


First Frame of Uploaded Video
Time: 0.0s
Time: 0.5s
Time: 1.0s
Time: 1.5s
Time: 2.0s
Time: 2.5s
Time: 3.0s
Time: 3.5s
Time: 4.0s
Time: 4.5s

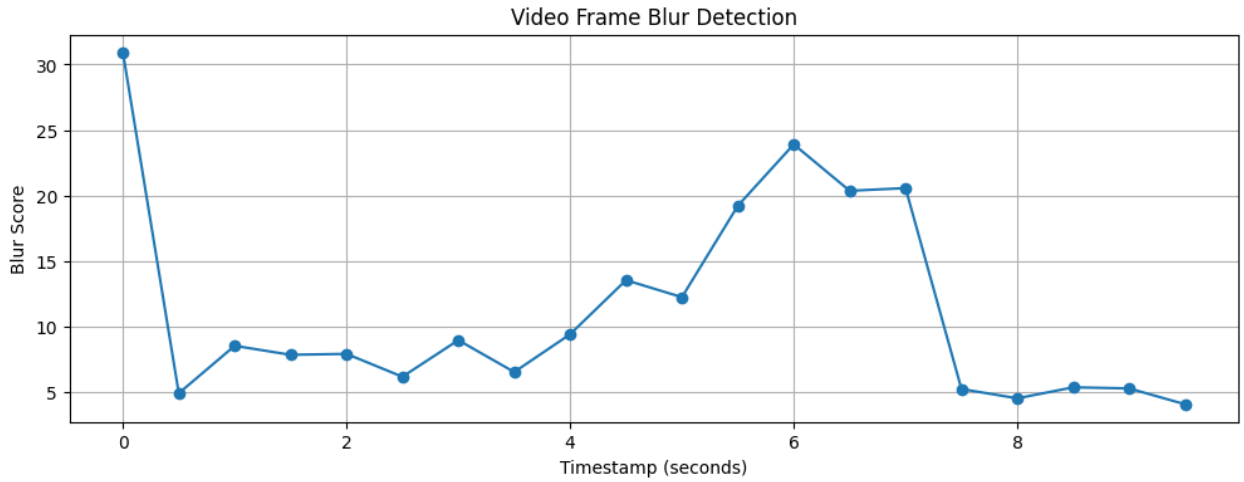
Blur detection

```
plt.figure(figsize=(12, 4))

plt.plot(timestamps, blur_scores, marker="o")
plt.xlabel("Timestamp (seconds)")
plt.ylabel("Blur Score")
plt.title("Video Frame Blur Detection")
plt.grid(True)

plt.show()

print("Lowest blur score:", round(min(blur_scores), 2))
print("Highest blur score:", round(max(blur_scores), 2))
```




```
Lowest blur score: 4.1
Highest blur score: 30.94
```

```
results = find_moment(
    "the girl is opening a door",
    top_k=5
)
```

```
Search query: the girl is opening a door
Top 5 matching frames:

Rank 1 | Timestamp: 9.50s | Similarity score: 31.804 | Blur score: 4.10
Rank 2 | Timestamp: 3.00s | Similarity score: 30.479 | Blur score: 8.98
Rank 3 | Timestamp: 2.00s | Similarity score: 30.479 | Blur score: 7.94
Rank 4 | Timestamp: 1.50s | Similarity score: 30.377 | Blur score: 7.87
Rank 5 | Timestamp: 2.50s | Similarity score: 30.076 | Blur score: 6.19
```

Natural-language matching and ranking

```
import matplotlib.pyplot as plt
import math

def display_results(results):
    if not results:
        print("No results to display.")
        return

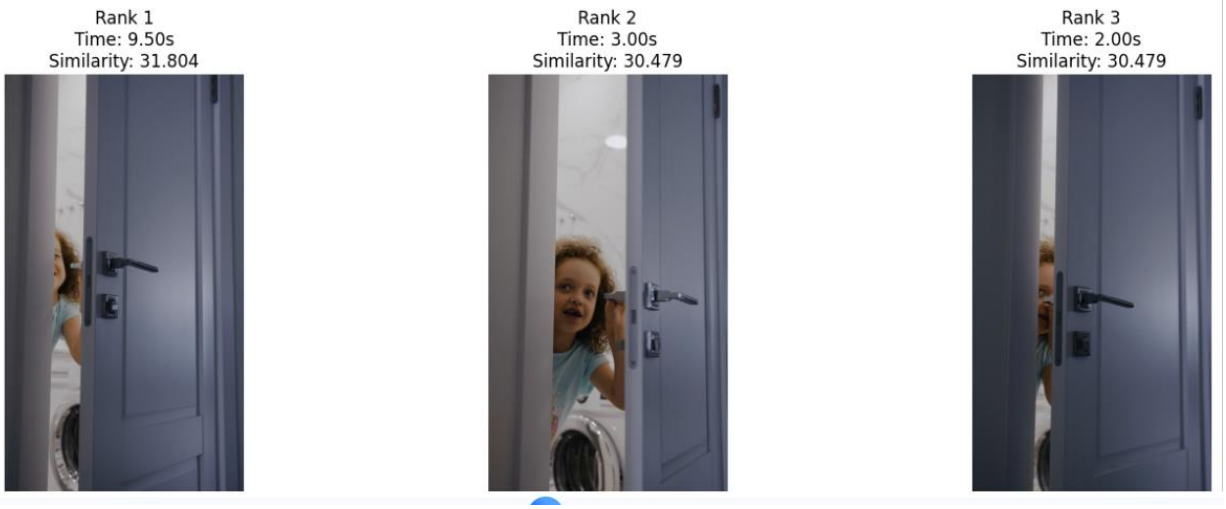
    n = len(results)
    cols = min(3, n)
    rows = math.ceil(n / cols)

    plt.figure(figsize=(15, 5 * rows))

    for i, result in enumerate(results):
        plt.subplot(rows, cols, i + 1)
        plt.imshow(result["image"])

        plt.title(
            f'Rank {result["rank"]}\n'
            f'Time: {result["timestamp"]:.2f}s\n'
            f'Similarity: {result["similarity"]:.3f}'
        )
```

```
display_results(results)
```





Flask backend

```
import flask
import cv2
import torch
import transformers

print("Flask version:", flask.__version__)
print("OpenCV version:", cv2.__version__)
print("PyTorch version:", torch.__version__)
print("CLIP model ready:", model is not None)
print("Device:", device)
```

```
Flask version: 3.1.3
OpenCV version: 4.14.0
PyTorch version: 2.11.0+cu130
CLIP model ready: True
Device: cuda
/tmp/ipykernel_1842/809124468.py:6: DeprecationWarning: The '__version__' attribute is deprecated and will be removed in Flask 3.2. Use feature detectio
  print("Flask version:", flask.__version__)
```

```
%%writefile /content/MomentMind_AI/app.py

import os
import cv2
import torch
import numpy as np

from flask import Flask, render_template, request
from PIL import Image
from transformers import CLIPModel, CLIPProcessor
from werkzeug.utils import secure_filename

app = Flask(__name__)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")
OUTPUT_FOLDER = os.path.join(BASE_DIR, "outputs")

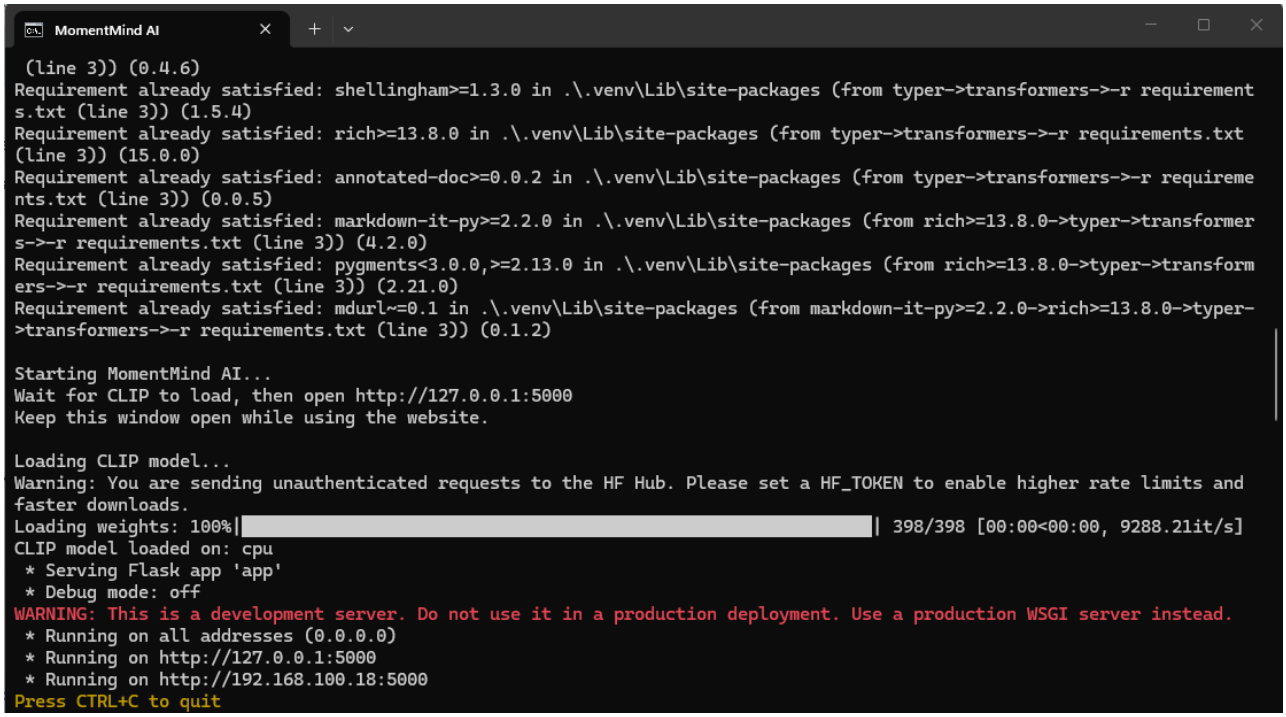
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(OUTPUT_FOLDER, exist_ok=True)

app.config["MAX_CONTENT_LENGTH"] = 200 * 1024 * 1024

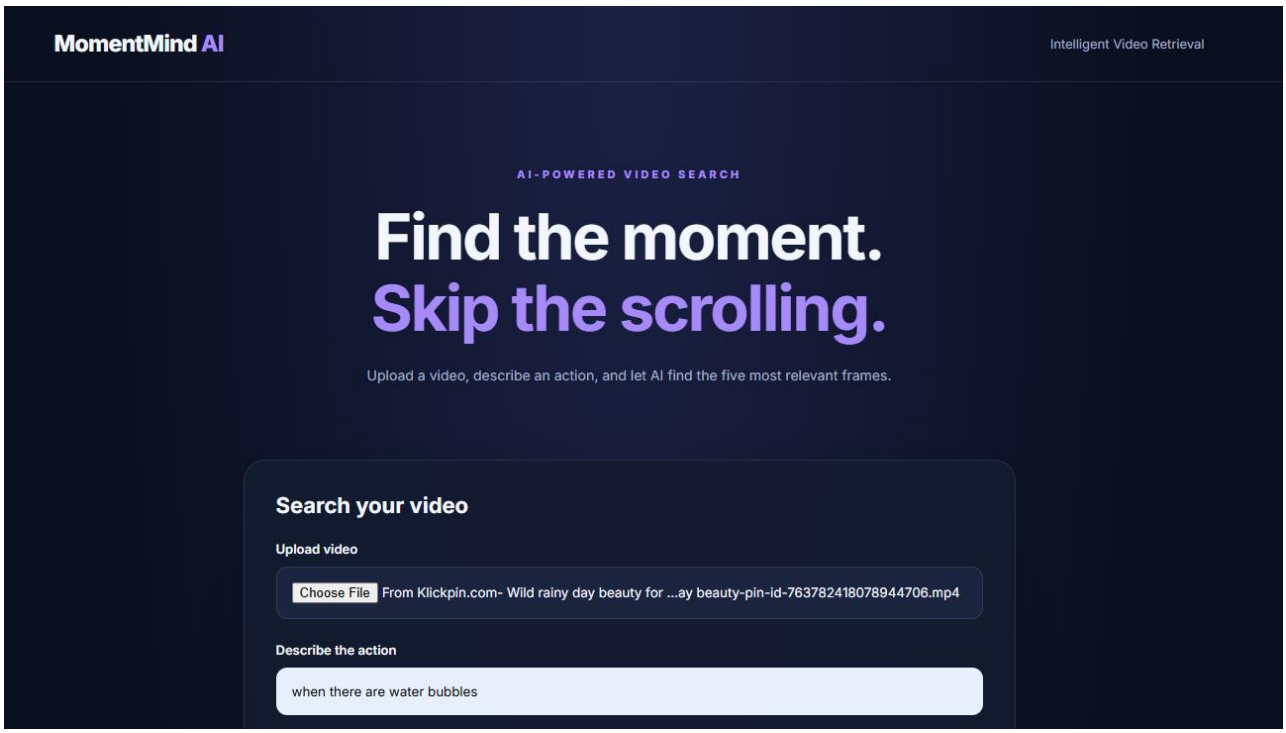
ALLOWED_EXTENSIONS = {"mp4", "mov", "avi", "mkv"}

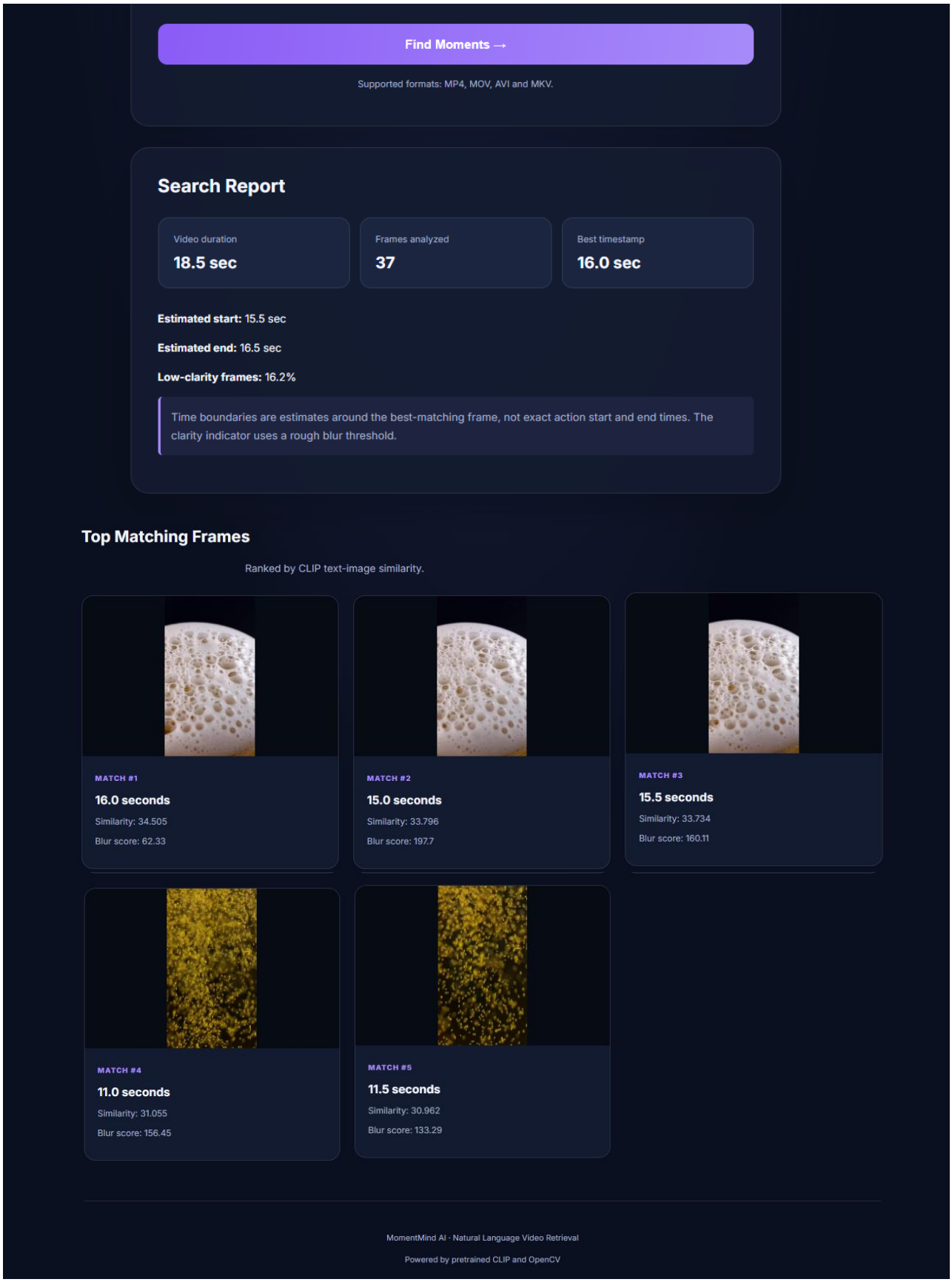
device = "cuda" if torch.cuda.is_available() else "cpu"
```

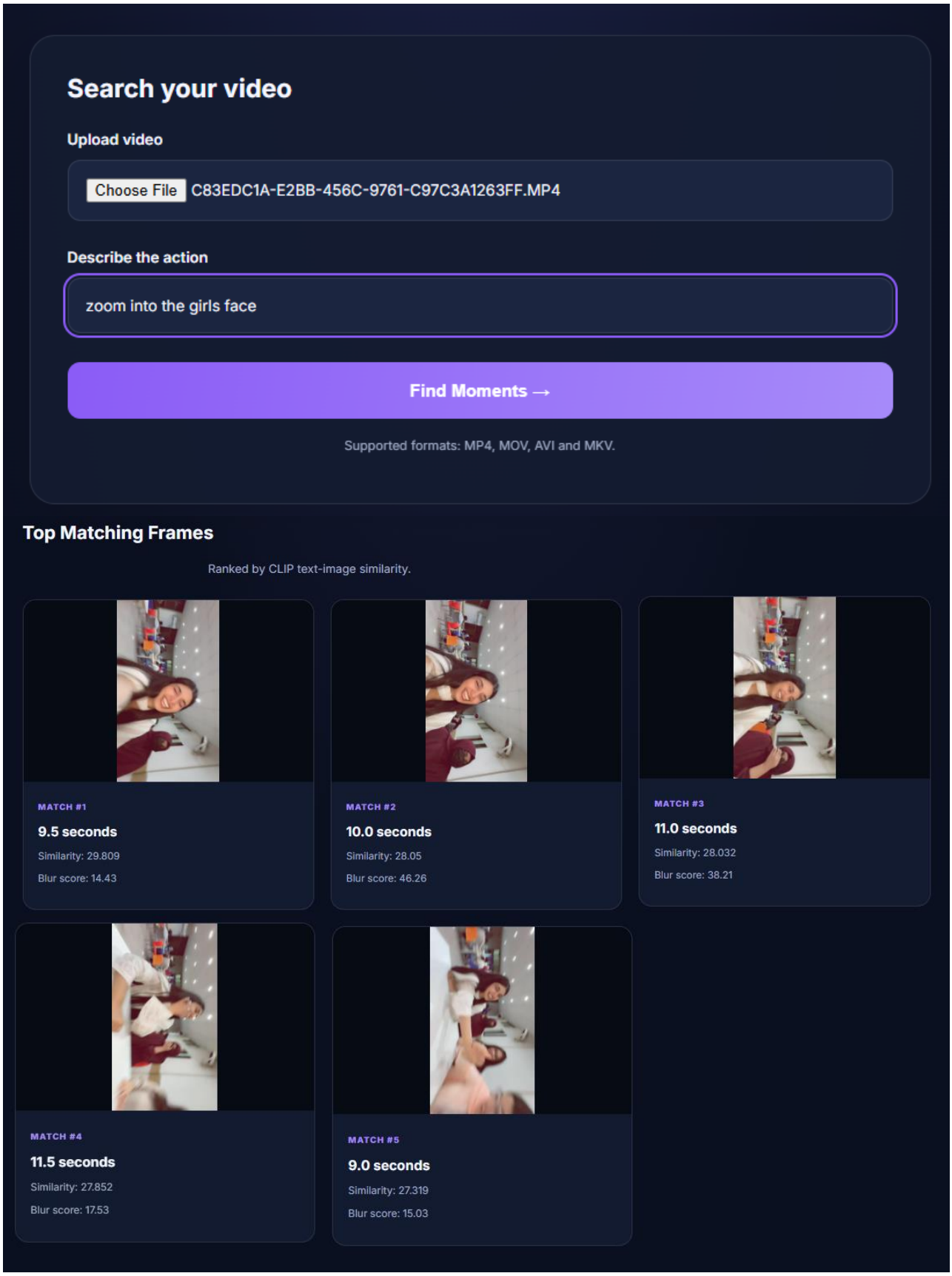
```
ZIP created successfully: /content/MomentMind_AI_Windows.zip
Files included:
✓ app.py
✓ templates/index.html
✓ static/style.css
✓ requirements.txt
✓ run_windows.bat
✓ README.txt
```



10. Moment Mind AI Homepage and Video Upload Interface




Find Moments →
Supported formats: MP4, MOV, AVI and MKV.
Search Report
Video duration
18.5 sec
Frames analyzed
37
Best timestamp
16.0 sec
Estimated start: 15.5 sec
Estimated end: 16.5 sec
Low-clarity frames: 16.2%
Time boundaries are estimates around the best-matching frame, not exact action start and end times. The clarity indicator uses a rough blur threshold.
Top Matching Frames
Ranked by CLIP text-image similarity.
MATCH #1
16.0 seconds
Similarity: 34.505
Blur score: 62.33
MATCH #2
15.0 seconds
Similarity: 33.796
Blur score: 197.7
MATCH #3
15.5 seconds
Similarity: 33.734
Blur score: 160.11
MATCH #4
11.0 seconds
Similarity: 31.055
Blur score: 156.45
MATCH #5
11.5 seconds
Similarity: 30.962
Blur score: 133.29
MomentMind AI · Natural Language Video Retrieval
Powered by pretrained CLIP and OpenCV


Search your video
Upload video
Choose File
C83EDC1A-E2BB-456C-9761-C97C3A1263FF.MP4
Describe the action
zoom into the girls face
Find Moments →
Supported formats: MP4, MOV, AVI and MKV.
Top Matching Frames
Ranked by CLIP text-image similarity.
MATCH #1
9.5 seconds
Similarity: 29.809
Blur score: 14.43
MATCH #2
10.0 seconds
Similarity: 28.05
Blur score: 46.26
MATCH #3
11.0 seconds
Similarity: 28.032
Blur score: 38.21
MATCH #4
11.5 seconds
Similarity: 27.852
Blur score: 17.53
MATCH #5
9.0 seconds
Similarity: 27.319
Blur score: 15.03

GitHub Repository Link:

https://github.com/jalishakhalid12-bit/MOMENT-MIND-AI-PROJECT-REPORT-24108209-